In this chapter, you'll see:

- How good matchers make for readable specs and helpful output
- How to define new matchers in terms of RSpec's built-in ones
- How to use RSpec's DSL to make a new matcher
- What a custom matcher class written by hand looks like

CHAPTER 12

Creating Custom Matchers

In the previous chapter, we took a tour of the matchers that ship with RSpec. You can be productive in RSpec with just these matchers. On simpler projects, they're all you'll need.

Eventually, though, you're going to hit the limits of the built-in matchers. Because they're meant for testing general-purpose Ruby code, they require you to speak in Ruby terms rather than your project's terms.

For example, the following expectations are a jumble of Ruby method calls and hard-coded values:

12-creating-custom-matchers/01/custom_matcher/spec/event_spec.rb

```
expect(art_show.tickets_sold.count).to eq(0)
expect(u2_concert.tickets_sold.count).to eq(u2_concert.capacity)
```

It takes a lot of reading and parsing, plus understanding the ticketing API we're testing, to understand exactly what behavior we're looking for. By contrast, the following snippet expresses the same desired behavior in much clearer terms:

12-creating-custom-matchers/01/custom_matcher/spec/event_spec.rb

```
expect(art_show).to have_no_tickets_sold
expect(u2_concert).to be_sold_out
```

Much, much better. We've added two *custom matchers*—`have_no_tickets_sold` and `be_sold_out`—so that we can describe the behavior in terms of *events* and *tickets*. These are the terms that the rest of the project team would use.

When you write clear, easy-to-use custom matchers, you gain several benefits:

- You stand a greater chance of building what your stakeholders want.
- You reduce the cost of API changes (because you need only update your matchers).
- You can provide better failure messages when something goes wrong.